TECHNICAL DOCUMENTATION & ENGINEERING STANDARDS

# Python Development Standards & Best Practices Guide

Official Python Language Reference, Environment Isolation, Code Standards, and Tooling

| | | | |
|---|---|---|---|
| **Standard:** Python 3.11 / 3.12 LTS | **Version:** 2.8 | **Owner:** Core Architecture Group | **Source Reference:** Python 3 Documentation |

## 1. Purpose and Scope

This document establishes the official enterprise engineering guidelines for writing, testing, packaging, and maintaining Python applications across Apex Global Technologies. All backend microservices, data pipelines, automation scripts, and machine learning components must comply with these standards.

## 2. Python Runtime and Virtual Environment Management

To avoid global package pollution, dependency collisions, and environment drift across development and production environments, **all Python projects must execute within an isolated virtual environment (venv)**.

### 2.1 Creating and Activating a Virtual Environment

Apex projects standardize on the standard library `venv` module. Follow this canonical procedure when setting up any Python repository:

```
# Step 1: Navigate to project root directory
cd ~/projects/enterprise-backend

# Step 2: Create a virtual environment named .venv using the standard Python 3 interpreter
python3 -m venv .venv

# Step 3: Activate the virtual environment
# On POSIX systems (Linux / macOS):
source .venv/bin/activate

# On Windows (PowerShell):
.\.venv\Scripts\Activate.ps1

# On Windows (Command Prompt):
.\.venv\Scripts\activate.bat

# Step 4: Verify the active interpreter points to the local virtual environment
which python   # Expected output: .../enterprise-backend/.venv/bin/python
python --version   # Must output Python 3.11.x or 3.12.x
```

> **Deactivation Command**
> To exit the isolated virtual environment at any time, execute the shell command: `deactivate`.

### 2.2 Dependency Pinning and Package Management

All production dependencies must be strictly locked with cryptographic hashes using `pip-tools` or pinned `requirements.txt`:

```
# Upgrade core packaging tools within the active virtual environment
python -m pip install --upgrade pip setuptools wheel

# Install production dependencies
pip install -r requirements.txt

# Install development and testing dependencies
pip install -r requirements-dev.txt
```

## 3. Code Style, Linting, and Formatting Standards

Apex enforces strict code uniformity across all repositories. Automated CI checks will reject pull requests that fail formatting gates:

- **PEP 8 Compliance:** Strict adherence to Python Enhancement Proposal 8 (Style Guide for Python Code).
- **Ruff Formatter & Linter:** Apex uses `ruff` as our ultra-fast universal Python linter and formatter. Maximum line length is locked at **88 characters** (consistent with Black conventions).
- **Import Sorting:** Standardize imports using `isort` rules built into Ruff: standard library first, third-party packages second, internal local modules third, each separated by a blank line.
- **Type Annotations (PEP 484 / PEP 526):** All function signatures, method parameters, and return types must include explicit type annotations. Repositories enforce `mypy --strict` in CI pipelines.

```
# Example of compliant enterprise Python code with strict typing
from typing import Optional, List
from dataclasses import dataclass
import logging

logger = logging.getLogger(__name__)

@dataclass(frozen=True)
class KnowledgeChunk:
    chunk_id: str
    document_id: str
    text_content: str
    embedding_score: float

def retrieve_top_chunks(query_vector: List[float], top_k: int = 5) -> List[KnowledgeChunk]:
    if not query_vector:
        logger.warning("Empty query vector supplied to retrieve_top_chunks.")
        return []
    # Vector retrieval execution logic
    return []
```

## 4. Testing, Quality Assurance, and Logging

- **Pytest Framework:** All automated tests must be authored using `pytest`. Test files must follow the naming pattern `test_*.py` located in a top-level `tests/` directory.
- **Coverage Floor:** Pull requests must maintain a minimum of **85% line and branch test coverage** verified via `pytest --cov=src --cov-fail-under=85`.
- **Structured JSON Logging:** Raw `print()` statements are forbidden in production code. Applications must configure Python's standard `logging` module to emit structured JSON logs with correlation IDs (`request_id`).